Advanced Concepts in CNN: Image Augmentation, TensorBoard, Model Saving, and Role of Bias

Image Augmentation

Image augmentation is a technique used to artificially increase the size and diversity of an image dataset by applying various transformations. This helps improve the robustness and generalization of machine learning models, especially when working with limited data.

---

1. Rotation

- **Description**: Rotating the image by a specific angle (e.g., 90°, 180°).
- **Advantages**:
  - Helps model learn orientation invariance.
  - Increases dataset diversity.
- **Disadvantages**:
  - Can distort the image if rotated excessively.
  - May introduce empty areas (padding).

---

2. Horizontal & Vertical Flipping

- **Description**: Flipping the image across the horizontal or vertical axis.
- **Advantages**:
  - Simple way to double the dataset size.
  - Encourages symmetry recognition.
- **Disadvantages**:
  - Unsuitable when object direction matters (e.g., text).

---

3. Zooming

- **Description**: Zooming in or out of the image.
- **Advantages**:
  - Simulates viewing the object at various distances.
  - Helps achieve scale invariance.
- **Disadvantages**:

- May crop out essential parts of the image.

---

4. Shifting (Translation)

- **Description**: Moving the image along the X or Y axis.
- **Advantages**:
  - Makes model less sensitive to object position.
- **Disadvantages**:
  - Parts of the object may be cut off if shifted too much.

---

5. Shearing

- **Description**: Slanting the image to create a skewed effect.
- **Advantages**:
  - Simulates 3D perspective changes.
- **Disadvantages**:
  - Can lead to unrealistic distortion.

---

6. Brightness Adjustment

- **Description**: Changing the image brightness.
- **Advantages**:
  - Helps model adapt to varying lighting conditions.
- **Disadvantages**:
  - Excessive brightness may hide key features.

---

7. Contrast Adjustment

- **Description**: Modifying the contrast level.
- **Advantages**:
  - Enhances object detection under shadow or strong lighting.
- **Disadvantages**:
  - High contrast can lead to loss of image detail.

---

8. Zoom Augmentation

- **Description**: Zooming into a specific part of the image.
- **Advantages**:

    - Helps model learn scale variations.
- **Disadvantages**:
    - Risk of removing critical image content.

---

9. Gaussian Noise

- **Description**: Adding random noise to pixels.
- **Advantages**:
    - Increases robustness to noisy environments.
- **Disadvantages**:
    - Excessive noise can obscure important features.

---

10. Gaussian Blur

- **Description**: Softening the image using a Gaussian filter.
- **Advantages**:
    - Encourages focus on broader patterns over fine details.
- **Disadvantages**:
    - May cause loss of sharpness and important features.

---

11. Cutout (Occlusion)

- **Description**: Randomly cutting out a section of the image.
- **Advantages**:
    - Trains model to handle occlusions.
- **Disadvantages**:
    - Can remove essential image parts.

---

12. Random Cropping

- **Description**: Cropping random portions of the image.
- **Advantages**:
    - Improves model's ability to recognize partial views.
- **Disadvantages**:
    - Key objects might be cropped out.

Overall, augmentation techniques are critical for training deep learning models with improved generalization, especially in image classification and object detection tasks.

TensorBoard & Model Saving in TensorFlow

🧠 What is TensorBoard?

TensorBoard is a powerful visualization toolkit provided by TensorFlow that allows you to monitor and debug the training process of machine learning models. It provides a visual interface to analyze metrics like **loss**, **accuracy**, **weights**, **biases**, and more.

📊 Key Features of TensorBoard

1. Scalars Dashboard

- Displays scalar values like **loss** and **accuracy** over time as line graphs.
- Useful for tracking training progress and detecting **overfitting** or **underfitting**.

2. Graph Visualization

- Visualizes the **computational graph** of the model.
- Helps in understanding the **model architecture** and flow of data through layers.

3. Histograms and Distributions

- Visualizes the distribution of weights, biases, and other tensors over time.
- Helps analyze how parameters evolve during training.

4. Image Visualization

- Displays input images, generated images, or feature maps during training.
- Useful for checking how the model perceives visual data.

5. Text Dashboard

- Logs textual information such as:
  - Custom messages
  - Model summaries
  - Training details

6. Embedding Visualization

- Projects high-dimensional data (like word embeddings or feature vectors) into a lower-dimensional space for visualization.
- Helps in understanding learned feature representations.

7. Audio Dashboard

- Allows playback and visualization of **audio data**, useful in speech and sound-based models.

8. Custom Scalars & Hyperparameters

- Log custom scalars like **learning rate**, **hyperparameters**, etc.
- Enables custom plots for deeper insight.

✅ Advantages of TensorBoard

- Real-time training **monitoring**
- Easier **debugging and optimization**
- Enhanced **collaboration** with visual insights
- Better understanding of **model behavior**

💾 Model Saving in TensorFlow

Feature	`model.save()`	`model.save_weights()`
Saves model architecture	✅ Yes	❌ No
Saves model weights	✅ Yes	✅ Yes
Saves optimizer state	✅ Yes	❌ No
File size	Larger (includes everything)	Smaller (weights only)
Use case	Deployment, sharing, resuming	Checkpointing, transfer learning
Required for loading	No code needed (self-contained)	Must define model architecture again

---

🕹️ When to Use What?

🔸 `model.save()`

- Use when you want to **save the complete model** (architecture + weights + optimizer state).
- Ideal for:
  - **Deployment**
  - **Sharing** the model

    - Resuming training without redefining architecture

🔸 `model.save_weights()`

- Use when you only need to save the **weights**, not the architecture.
- Ideal for:
    - **Checkpointing** during training
    - **Transfer learning** (reuse weights with new models)
    - Experimenting with different architectures using same weights

TensorBoard & Model Saving in TensorFlow

🧠 What is TensorBoard?

TensorBoard is a powerful visualization toolkit provided by TensorFlow that allows you to monitor and debug the training process of machine learning models. It provides a visual interface to analyze metrics like **loss**, **accuracy**, **weights**, **biases**, and more.

📊 Key Features of TensorBoard

1. Scalars Dashboard

- Displays scalar values like **loss** and **accuracy** over time as line graphs.
- Useful for tracking training progress and detecting **overfitting** or **underfitting**.

2. Graph Visualization

- Visualizes the **computational graph** of the model.
- Helps in understanding the **model architecture** and flow of data through layers.

3. Histograms and Distributions

- Visualizes the distribution of weights, biases, and other tensors over time.
- Helps analyze how parameters evolve during training.

4. Image Visualization

- Displays input images, generated images, or feature maps during training.
- Useful for checking how the model perceives visual data.

5. Text Dashboard

- Logs textual information such as:
  - Custom messages
  - Model summaries
  - Training details

6. Embedding Visualization

- Projects high-dimensional data (like word embeddings or feature vectors) into a lower-dimensional space for visualization.
- Helps in understanding learned feature representations.

7. Audio Dashboard

- Allows playback and visualization of **audio data**, useful in speech and sound-based models.

8. Custom Scalars & Hyperparameters

- Log custom scalars like **learning rate**, **hyperparameters**, etc.
- Enables custom plots for deeper insight.

✅ Advantages of TensorBoard

- Real-time training **monitoring**
- Easier **debugging and optimization**
- Enhanced **collaboration** with visual insights
- Better understanding of **model behavior**

💾 Model Saving in TensorFlow

Feature	`model.save()`	`model.save_weights()`
Saves model architecture	✅ Yes	❌ No
Saves model weights	✅ Yes	✅ Yes
Saves optimizer state	✅ Yes	❌ No
File size	Larger (includes everything)	Smaller (weights only)
Use case	Deployment, sharing, resuming	Checkpointing, transfer learning
Required for loading	No code needed (self-contained)	Must define model architecture again

---

🕹️ When to Use What?

🔸 `model.save()`

- Use when you want to **save the complete model** (architecture + weights + optimizer state).
- Ideal for:
  - **Deployment**
  - **Sharing** the model
  - Resuming training without redefining architecture

🔸 `model.save_weights()`

- Use when you only need to save the **weights**, not the architecture.
- Ideal for:
  - **Checkpointing** during training
  - **Transfer learning** (reuse weights with new models)
  - Experimenting with different architectures using same weights

💡 Example Use Cases

1. **Deploying a Model**

   ```
   model.save("model_full.h5")
   ```
2. **Saving Checkpoints During Training**

   ```
   model.save_weights("checkpoint_weights.h5")
   ```
3. **Transfer Learning**

   - Save pretrained weights:

   ```
   model.save_weights("pretrained_weights.h5")
   ```

   - Load them into another model with the same architecture:

   ```
   new_model.load_weights("pretrained_weights.h5")
   ```

🧠 Role and Importance of **Bias** in Convolutional Neural Networks (CNNs)

Bias in a neural network is an additional trainable parameter added to the output of each neuron (after the weighted sum of inputs). In **CNNs**, it plays a critical role in enabling the model to learn flexible and complex patterns beyond direct input-output proportionality.

🎯 Role of Bias in CNNs

1. Shifting the Activation Function

- Bias allows the **activation function** to shift **left or right**.
- Without bias, a neuron's output is entirely determined by the weighted input sum.
- This shift enables the network to learn patterns **even when the input is zero** or **features are missing**.

2. Avoiding Restriction of Output

- Without bias, output = weighted sum of inputs → too restrictive.
- For **non-linear and complex patterns**, this restriction reduces the network's learning ability.
- Bias introduces **flexibility**, allowing the network to learn more generalized functions.

3. Learning Intercepts (Like in Line Equation)

- Equation: `y = mx + b`
  - `m` → slope (weights)
  - `b` → intercept (bias)
- In CNNs, bias helps **shift the decision boundary**.
- Essential in **classification tasks**, allowing neurons to activate under **non-centered** or **shifted features**.

4. Enhancing Flexibility and Generalization

- Bias allows each neuron to have its own **activation threshold**.
- This makes the model **adaptable** to a wide variety of data distributions.
- Critical in image-based tasks where patterns:
  - Appear in **varied locations**
  - May not be **uniformly distributed**

📌 Importance of Bias in CNNs

🔸 Learning Complex Patterns

- Bias enables neurons in convolution layers to:
  - Detect features that don't originate from the origin
  - Adapt to **shifted or irregular patterns**
- Vital for learning from **real-world images**, where objects vary in shape and position.

🔸 Improving Training Stability

- Bias provides **additional degrees of freedom** for optimization.
- Prevents the model from **getting stuck** during training when weights alone can't fit the data well.

🔸 Enabling Robust Feature Detection

- Each neuron detects different features.
- Bias allows neurons to adjust their **sensitivity** to these features.
- Leads to **more robust and reliable learning** across training samples.

✅ Summary

Without Bias	With Bias
Restricted output	Flexible output
No shift in activation	Shifts activation function
Poor generalization	Better at capturing varied patterns
Struggles with complex data	Learns intercepts and robustly fits

Bias is **essential** in CNNs for learning rich, non-linear representations from image data, enabling the model to be **flexible**, **robust**, and **generalizable**.

In [2]:
```
# # This Python 3 environment comes with many helpful analytics libraries installed
# # It is defined by the kaggle/python Docker image: https://github.com/kaggle/dock
# # For example, here's several helpful packages to load

# import numpy as np # linear algebra
# import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# # Input data files are available in the read-only "../input/" directory
# # For example, running this (by clicking run or pressing Shift+Enter) will list a

# import os
# for dirname, _, filenames in os.walk('/kaggle/input'):
#     for filename in filenames:
#         print(os.path.join(dirname, filename))

# # You can write up to 20GB to the current directory (/kaggle/working/) that gets
# # You can also write temporary files to /kaggle/temp/, but they won't be saved ou
```

In [1]:
```
import os
import numpy as np
import pandas as pd
import random as rn
import warnings
warnings.filterwarnings("ignore")
```

```
import tensorflow as tf
import keras

from tensorflow.keras.applications.vgg16 import VGG16,preprocess_input
from tensorflow.keras.models import Sequential

from tensorflow.keras.models import Model, load_model,Sequential
from tensorflow.keras.layers import Input, Dense,Conv2D,MaxPool2D,Activation,Flatte
from tensorflow.keras.callbacks import ModelCheckpoint, TensorBoard
from tensorflow.keras import regularizers
```

```
2025-07-25 17:06:30.251732: E external/local_xla/xla/stream_executor/cuda/cuda_fft.c
c:477] Unable to register cuFFT factory: Attempting to register factory for plugin c
uFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDE
RR
E0000 00:00:1753463190.609907      36 cuda_dnn.cc:8310] Unable to register cuDNN fac
tory: Attempting to register factory for plugin cuDNN when one has already been regi
stered
E0000 00:00:1753463190.718880      36 cuda_blas.cc:1418] Unable to register cuBLAS f
actory: Attempting to register factory for plugin cuBLAS when one has already been r
egistered
```

In [2]:
```
dir_path = '/kaggle/input/rvlcdip/'
os.listdir(dir_path)
```

Out[2]:
```
['labels_final.csv', 'data_final']
```

In [3]:
```
traindf = pd.read_csv('/kaggle/input/rvlcdip/labels_final.csv')
traindf.head()
```

Out[3]:

	path	label
0	imagesv/v/o/h/voh71d00/509132755+-2755.tif	3
1	imagesl/l/x/t/lxt19d00/502213303.tif	3
2	imagesx/x/e/d/xed05a00/2075325674.tif	2
3	imageso/o/j/b/ojb60d00/517511301+-1301.tif	3
4	imagesq/q/z/k/qzk17e00/2031320195.tif	7

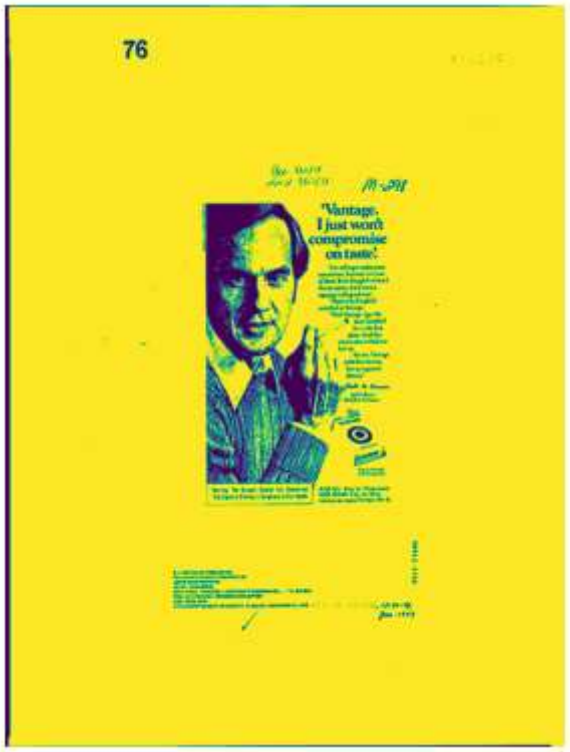
In [4]:
```
traindf['path'].head()
```

Out[4]:
```
0    imagesv/v/o/h/voh71d00/509132755+-2755.tif
1          imagesl/l/x/t/lxt19d00/502213303.tif
2         imagesx/x/e/d/xed05a00/2075325674.tif
3    imageso/o/j/b/ojb60d00/517511301+-1301.tif
4         imagesq/q/z/k/qzk17e00/2031320195.tif
Name: path, dtype: object
```

In [5]:
```
from PIL import Image
import matplotlib.pyplot as plt
```

```
# Load and display a .tif image
image_path = '/kaggle/input/rvlcdip/data_final/imagesv/v/o/h/voh71d00/509132755+-27
image = Image.open(image_path)

plt.imshow(image)
plt.axis('off')  # Turn off axis labels
plt.show()
```



In [6]:
```
traindf['label'].value_counts()
```

Out[6]:
```
label
0     3016
13    3007
14    3006
12    3006
3     3005
8     3003
9     3002
10    3002
7     3000
5     2999
15    2996
1     2994
4     2994
2     2993
11    2992
6     2985
Name: count, dtype: int64
```

In [7]:
```
datagen = tf.keras.preprocessing.image.ImageDataGenerator(
                                                          rescale = 1./255,
                                                          rotation_range = 10,
```

```
                                                    width_shift_range = 0.1
                                                    height_shift_range = 0.
                                                    shear_range = 0.1,
                                                    zoom_range = 0.1,
                                                    horizontal_flip = True,
                                                    validation_split = 0.2)
```

In [8]:
```
train_generator = datagen.flow_from_dataframe(dataframe= traindf,
                                              directory='/kaggle/input/rvlcdip/data
                                              x_col = 'path',
                                              y_col = 'label',
                                              subset = 'training',
                                              batch_size=100,
                                              seed=42,
                                              shuffle=True,
                                              class_mode="raw",
                                              target_size=(224,224))
```

```
Found 38400 validated image filenames.
```

While passing the data to traininf, it will do data aug

It laoded the augemented images during the batch.

In [9]:
```
images, labels = next(train_generator)
print(len(images), len(labels))
```

```
100 100
```

In [10]:
```
print(labels)
```

```
[ 9 11  8  8  4  3  0  1  2  7  0  6 13  7  7 15  2 14  6  6  2  7 10  6
 10  1 13  7 13  8 11 10  7 13  8  1  5 10  6 14 15  2 10  0  0 12 15  2
 13 11  6 13  5 12 15  8 13 13 14 14 13  0  2 13  4 10 10  6  9  9 13 11
  0  6  3 15 11  8 14  0  6  1  8  1 12  8  8 14 14  5 10 10 11  0 13 15
  2  0  3  8]
```

In [11]:
```
valid_generator = datagen.flow_from_dataframe(dataframe= traindf,
                                              directory='/kaggle/input/rvlcdip/data
                                              x_col = 'path',
                                              y_col = 'label',
                                              subset = 'validation',
                                              batch_size=100,
                                              seed=42,
                                              shuffle=True,
                                              class_mode="raw",
                                              target_size=(224,224))
```

```
Found 9600 validated image filenames.
```

Custom CNN Model from Scratch

In [12]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential([
    Conv2D(32, (3,3), activation = 'relu', input_shape=(224, 224,3)),
    MaxPooling2D(pool_size=(2,2)),

    Conv2D(64, (3,3), activation = 'relu'),
    MaxPooling2D(pool_size=(2,2)),

    Conv2D(128, (3,3), activation = 'relu'),
    MaxPooling2D(pool_size=(2,2)),

    Flatten(),
    Dense(128, activation = 'relu'),
    Dense(64, activation = 'relu'),
    Dense(16, activation = 'softmax'), # 16 classes probability of all 16 classes.
])
```

```
I0000 00:00:1753463598.022464      36 gpu_device.cc:2022] Created device /job:localh
ost/replica:0/task:0/device:GPU:0 with 13942 MB memory:  -> device: 0, name: Tesla T
4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1753463598.023179      36 gpu_device.cc:2022] Created device /job:localh
ost/replica:0/task:0/device:GPU:1 with 13942 MB memory:  -> device: 1, name: Tesla T
4, pci bus id: 0000:00:05.0, compute capability: 7.5
```

In [13]:
```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 128)	11,075,712
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 16)	1,040

Total params: 11,178,256 (42.64 MB)

Trainable params: 11,178,256 (42.64 MB)

Non-trainable params: 0 (0.00 B)

In [14]:
```
from tensorflow.keras.callbacks import ModelCheckpoint, ReduceLROnPlateau,EarlyStop
import os
import datetime
```

In [15]:
```
checkpoint_path = "kaggle/working/model_checkpoints/weights.{epoch:02d}-{val_loss:2
log_dir = "/kaggle/working/logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H


# Epoch 1 - Weights.keras
# epoch 2 - Weights.keras

# Naming format - "weights.{epoch:02d}-{val_loss:2f}.keras"

model_checkpoint = ModelCheckpoint(filepath=checkpoint_path,
                                   save_best_only=True,
                                   monitor='val_loss',
                                   mode='min',
                                   verbose=1)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=3, min_lr=0.
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=Tru
board = TensorBoard(log_dir=log_dir, histogram_freq=1)

callbacks = [model_checkpoint, reduce_lr, early_stop, board]


# e1 - 0.7 ( check point )
# e2 - 0.71
```

```
# e3 - 0.72
# e4 - 0.75
# e5 - 0.8
# stop the training- restore my check point.

# reduce my lr
# e4 - 0.65
# when ever there is improvement, i want the model to be saved.
```

In [16]:
```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 128)	11,075,712
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 16)	1,040

Total params: 11,178,256 (42.64 MB)

Trainable params: 11,178,256 (42.64 MB)

Non-trainable params: 0 (0.00 B)

In [17]:
```
model.compile(optimizer = 'adam',
              loss= 'sparse_categorical_crossentropy',
              metrics = ['accuracy'])
```

In [ ]:
```
history = model.fit(
                    train_generator,
                    validation_data=valid_generator,
                    epochs=50,
                    callbacks=callbacks
                    )
```

Epoch 1/50

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDE
RR
I0000 00:00:1753463608.367930     106 service.cc:148] XLA service 0x7ce52800ee90 ini
tialized for platform CUDA (this does not guarantee that XLA will be used). Devices:
I0000 00:00:1753463608.369413     106 service.cc:156]   StreamExecutor device (0): T
esla T4, Compute Capability 7.5
I0000 00:00:1753463608.369435     106 service.cc:156]   StreamExecutor device (1): T
esla T4, Compute Capability 7.5
I0000 00:00:1753463608.759453     106 cuda_dnn.cc:529] Loaded cuDNN version 90300
I0000 00:00:1753463619.422057     106 device_compiler.h:188] Compiled cluster using
XLA!  This line is logged at most once for the lifetime of the process.
384/384 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.2261 - loss: 2.4403
Epoch 1: val_loss improved from inf to 1.82241, saving model to kaggle/working/model
_checkpoints/weights.01-1.822414.keras
384/384 ━━━━━━━━━━━━━━━━━━━━ 1112s 3s/step - accuracy: 0.2263 - loss: 2.4396 - val_a
ccuracy: 0.4350 - val_loss: 1.8224 - learning_rate: 0.0010
Epoch 2/50
384/384 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.4628 - loss: 1.7203
Epoch 2: val_loss improved from 1.82241 to 1.56358, saving model to kaggle/working/m
odel_checkpoints/weights.02-1.563579.keras
384/384 ━━━━━━━━━━━━━━━━━━━━ 831s 2s/step - accuracy: 0.4629 - loss: 1.7201 - val_ac
curacy: 0.5216 - val_loss: 1.5636 - learning_rate: 0.0010
Epoch 3/50
384/384 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.5482 - loss: 1.4804
Epoch 3: val_loss improved from 1.56358 to 1.39060, saving model to kaggle/working/m
odel_checkpoints/weights.03-1.390597.keras
384/384 ━━━━━━━━━━━━━━━━━━━━ 829s 2s/step - accuracy: 0.5483 - loss: 1.4803 - val_ac
curacy: 0.5693 - val_loss: 1.3906 - learning_rate: 0.0010
Epoch 4/50
384/384 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.5816 - loss: 1.3594
Epoch 4: val_loss improved from 1.39060 to 1.32905, saving model to kaggle/working/m
odel_checkpoints/weights.04-1.329048.keras
384/384 ━━━━━━━━━━━━━━━━━━━━ 822s 2s/step - accuracy: 0.5816 - loss: 1.3594 - val_ac
curacy: 0.5999 - val_loss: 1.3290 - learning_rate: 0.0010
Epoch 5/50
384/384 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.6099 - loss: 1.2672
Epoch 5: val_loss improved from 1.32905 to 1.28276, saving model to kaggle/working/m
odel_checkpoints/weights.05-1.282757.keras
384/384 ━━━━━━━━━━━━━━━━━━━━ 824s 2s/step - accuracy: 0.6099 - loss: 1.2672 - val_ac
curacy: 0.6101 - val_loss: 1.2828 - learning_rate: 0.0010
Epoch 6/50
384/384 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.6239 - loss: 1.2181
Epoch 6: val_loss improved from 1.28276 to 1.20513, saving model to kaggle/working/m
odel_checkpoints/weights.06-1.205132.keras
384/384 ━━━━━━━━━━━━━━━━━━━━ 815s 2s/step - accuracy: 0.6239 - loss: 1.2180 - val_ac
curacy: 0.6378 - val_loss: 1.2051 - learning_rate: 0.0010
Epoch 7/50
 21/384 ━━━━━━━━━━━━━━━━━━━━ 10:18 2s/step - accuracy: 0.6688 - loss: 1.1519
```

In [ ]: